



국민대학교
소프트웨어융합대학
소프트웨어학부

다학제간캡스톤디자인 종합설계프로젝트

프로젝트 명	TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기
팀 명	팀 2
문서 제목	최종 보고서

날짜	2026-06-13
----	------------

팀원	김신영 (조장)
	강민성
지도교수	김형균

제 출 문

본 보고서를 다학제간캡스톤디자인I 교과목의
종합설계 프로젝트 최종보고서로 제출합니다.

2026.06.19

프로젝트 명 : TraceInspector

팀 구성원 : 김신영(팀장)
강민성

지도교수 : 김형균 교수

목 차

1. 요약
2. 중간 보고서 발표자료
3. 결과 보고서
 - 3.1 결과 보고서
 - 3.2 결과 보고서 발표자료
4. 회의록

제 1 장

요약

코드 정적 분석은 프로그램을 실행하지 않고 코드를 논리적으로 분석하여 동적 특성을 파악하는 기법이며 런타임 오류, 주어진 명세 준수 등 다양한 논리적/의미적 성질을 검증하기 위해 사용된다. TracInspector는 요약해석(abstract interpretation)이론을 바탕으로 구현된 추적 가능한 시각화 기반 코드 정적분석기이다. 단순히 오류를 탐지해서 알려주는 정적 분석기를 넘어, 분석 과정을 시각적으로 나타내어 사용자가 어떤 과정을 통해 오류가 탐지되었는지 해석 가능하게끔 돕고자 한다.

제 2 장

중간 보고서 발표자료

제 3 장

결과보고서

3.1 결과보고서

3.2 결과보고서 발표자료



국민대학교
소프트웨어융합대학
소프트웨어학부

캡스톤디자인 I

종합설계프로젝트

프로젝트 명	TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기
팀 명	팀 2
문서 제목	결과보고서

Version	1.0
Date	2026-06-13

팀원	김신영 (조장)
	강민성



목 차

1	개요	2
1.1	프로젝트 개요	2
1.2	추진 배경 및 필요성	2
2	개발 내용 및 결과물	4
2.1	목표	4
2.2	연구/개발 내용 및 결과물	5
2.2.1	연구/개발 내용.....	5
2.2.1.1	TraceInspector 분석기	5
	배경: 정적 프로그램 분석과 결정가능성(decidability)	5
	배경: 프로그램 의미론	7
	배경: 요약해석(abstract interpretation)	8
	Imp 중간 표현 언어	13
	요약 도메인.....	14
	제어 흐름 그래프(control-flow graph).....	17
	요약실행 의미론.....	18
	오류 상태 판별	20
2.2.1.2	TraceInspector 프론트엔드	21
2.2.2	시스템 기능 요구사항.....	21
2.2.3	시스템 비기능(품질) 요구사항	21
2.2.4	시스템 구조 및 설계도.....	21
2.2.5	활용/개발된 기술	22
2.2.6	현실적 제한 요소 및 그 해결 방안	22
2.2.7	결과물 목록	22
2.3	기대효과 및 활용방안.....	24
3	자기평가	24
4	참고 문헌	24
5	부록	25
5.1	사용자 메뉴얼	25
5.2	운영자 메뉴얼	25
5.3	배포 가이드.....	25
5.4	Imp 형식 의미론.....	25
5.4.1	요약 도메인 Go 인터페이스	27

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

1 개요

1.1 프로젝트 개요

TraceInspector는 요약해석(abstract interpretation)에 기반한 정적 분석기이다.

1.2 추진 배경 및 필요성

올바르고 안전한 코드를 작성하는 것은 모든 프로그래머의 목표이다. 그러나 현대의 프로그래밍 환경은 고도의 추상화와 복잡한 소프트웨어 구성 요소들로 이루어져 있어, 처음부터 완전히 안전한 코드를 작성하는 것은 사실상 불가능에 가깝다. 대신 개발자는 코드를 작성하고, 오류를 탐지하며, 디버깅을 통해 문제를 수정하는 과정을 반복함으로써 점진적으로 프로그램의 안전성과 신뢰성을 향상시킨다.

이 과정에서 오류를 효과적으로 탐지하는 것은 매우 중요하다. 오류가 탐지되어야만 디버깅 과정이 시작될 수 있으며, 프로그램에 존재하는 문제의 원인과 위치를 파악할 수 있기 때문이다. 따라서 소프트웨어 개발 과정에서는 단위 테스트(unit testing), 통합 테스트(integration testing), 코드 리뷰(code review), 정적 분석(static analysis) 등 다양한 오류 탐지 기법들이 적극적으로 활용된다.

정적 프로그램 분석(static program analysis)이란 프로그램을 직접 실행하지 않고 소스 코드를 분석하여 실제 동작 특성을 추론하는 기법들을 의미한다.

```
infer@facebook:~/infer/examples$ infer -- javac Infer.java
Starting analysis (Infer version v0.5.0)
Computing dependencies... 100%
Analyzing 1 cluster. 100%
Analyzed 2 procedures in 1 file
No issues found
infer@facebook:~/infer/examples$ vim Infer.java
infer@facebook:~/infer/examples$ infer -- javac Infer.java
Starting analysis (Infer version v0.5.0)
Computing dependencies... 100%
Analyzing 1 cluster. 100%
Analyzed 3 procedures in 1 file

Found 1 issue
Infer.java:12: error: NULL_DEREFERENCE
object s last assigned on line 11 could be null and is dereferenced at line 12
10.     int mayCauseNPE() {
11.         String s = mayReturnNull(0);
12. >         return s.length();
13.     }
14.
```

그림 1: Infer 분석기를 이용해 Java 코드의 NullPointerException을 탐지하는 모습

IDE에서 코드를 저장할 때마다 표시되는 unused variable 등의 경보, 컴파일러에서 보다 효율적인 코드 생성을 위해 수행하는 dataflow analysis, CI/CD 절차에서 커밋 후 수행되는 code linter 등 개발의 전 과정에서 프로그래머가 알게 모르게 활용되고 있다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

Facebook Infer[2], clang static analyzer[6] 등 실제 프로그래머가 활용하여 오류를 탐지하고자 하는 목적으로 작성된 정적 분석기와, LLVM[7] 등의 컴파일러 내부에서 중간절차로 활용되는 경우와 같이 사용 환경과 분석 목표는 서로 다르지만, 공통적으로 다음과 같은 목표를 가진다:

- 정확성: 의미 있는 오류 정보를 정확하게 제공해야 한다.
- 신속성: 분석 과정이 신속해야 하며, 지나치게 많은 시공간 자원을 소모해서는 안된다.
- 편리성 및 자동화: 최소한의 사용자 개입 및 추가적인 정보 없이 독자적으로 동작해야 한다.

즉 대량의 코드에 대해 빠르고 정확하게 분석하는 것을 목표로 한다.

하지만 우리가 정적 분석기를 활용하면서, 한번쯤은 다음과 같은 질문들을 해볼 수 있다:

- 정적 분석기는 어떤 과정을 통해 코드 속의 오류를 탐지할 수 있을까?
- 이러한 정적 분석기의 결과를 우리가 신뢰할 수 있을까?
- 왜 "정적"일까?

이러한 질문들은 개발자에게 익숙한 일상적 경험으로 인해 간과되기 쉽다. 하지만 단순히 코드를 입력으로 넣었을 때 오류 정보를 알려주는 "마법의 상자"로 생각하지 않고, 그 내부의 추론 과정과 분석 원리를 이해할 수 있다면, 분석 결과를 보다 효과적으로 해석하고 활용할 수 있을 것이다.

TraceInspector는 위와 같은 궁금증을 해소하고자 하는 취지에서 개발되었다. 정적 프로그램 분석 기법 중 요약해석(abstract interpretation) 이론에 대해 소개하고, 단순히 어떤 오류가 존재하는지에 대한 정보를 제공하는 것을 넘어 "어떻게" 해당 이론을 통해 탐지되었는지 시각화하여 보여주하고자 한다. 이를 통해 프로그래머들에게 정적 분석 이론에 대한 이해와 친숙도를 높이고, 단순히 코드를 작성하는 행위를 넘어 논증의 대상으로 바라보는 관점을 소개하고자 한다.

알고리즘 혹은 자료구조 강의를 생각해보자. 이를테면 그래프 탐색 알고리즘을 처음 접했을 때, 알고리즘의 수학적 내용과 의사 코드만 보아서는 온전히 해당 개념을 직관적으로 이해하기 어렵다. 대신 실제 그래프 위에서 동작하는 예시를 통해 비로소 직관적인 이해가 가능해지는 경우가 많다. 특히나 알고리즘과 같이 추상화의 정도가 높은 주제의 경우 더 더욱 그럴 것이다. 마찬가지로 TraceInspector 역시 정적 분석 알고리즘을 시각화하여 동작 원리에 대해 와닿을 수 있는 설명을 제시하는 것이 목표이다.



2 개발 내용 및 결과물

2.1 목표

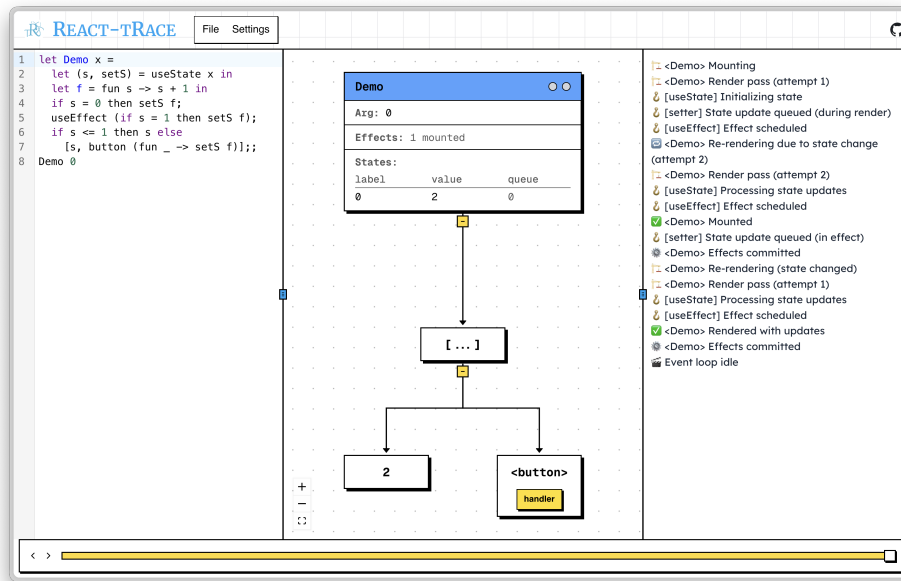


그림 2: react-tRace의 시각화 인터페이스

React-tRace[8]는 React 웹 프레임워크의 Hook가 어떤 과정을 따라 컴포넌트를 변화시키는 지 단계적인 형식 의미론을 정의한 연구이다. *React-tRace*의 흥미로운 부분은 기존 형식 의미론 연구와는 다르게 웹 인터페이스(그림 2)를 통해 사용자가 직접 해당 의미론에 따라 웹페이지 컴포넌트의 상태 변화를 직접 관찰할 수 있게 해준 점이다. 따라서 단순히 수식으로 정의한 의미론을 읽는 것을 넘어 각 변화 단계가 어떤 수식에 의해 변했는지 구체적으로 추론하고 이해할 수 있다는 점이 직관적으로 와닿고, 복잡한 알고리즘을 이해하기에 탁월한 방법임을 파악하게 되었다.

TraceInspector는 해당 연구로부터 영감을 받아 단순히 오류를 탐지해서 알려주는 정적 분석기를 넘어, 분석 과정을 시각적으로 나타내어 사용자가 어떤 과정을 통해 오류가 탐지되었는지 해석 가능하게끔 구현된 정적 분석기이다. 즉 분석기의 동작 흐름을 사용자가 직접 관찰하며 올바른 추론을 분석기가 수행했는지, 그리고 어떻게 정적 분석이 이루어졌는지를 전달하고자 본 프로젝트를 구상하였다. 다양한 오류를 신속하게 분석하는 게 아닌 어떤 과정으로 정적 분석이 프로그램을 처리하고 오류를 탐지하는지에 대한 정보 전달 및 시각화를 목표로 하며, 이에 따른 프로그래밍 언어론과 정적 분석 이론에 대한 교육적 자료로 활용되기를 바란다.

또한 *React-tRace*의 형식 의미론처럼 신뢰 가능하고 이론적으로 건전한 분석기를 구현하는 것을 목표로 하였다. 이에 따라 기존 정적 분석기보다는 높은 수준의 신뢰성을 추구할

 국민대학교 소프트웨어학부 다학제간캡스톤디자인 I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

수 있도록 설계하였고 요약해석 이론을 도입하여 엄밀한 명세와 정의를 바탕으로 정적 분석기를 구현하였다.

2.2 연구/개발 내용 및 결과물

2.2.1 연구/개발 내용

TraceInspector는 요약해석 이론을 구현하여 소스 코드를 분석하는 "분석기"와 분석과 관련된 정보를 시각화하여 나타내는 "인터페이스"로 구성되어 있다.

다음은 각 요소별 설명이다.

2.2.1.1 TraceInspector 분석기

TraceInspector 분석기는 Go 언어 소스 코드를 입력으로 받아, 다음의 정보를 출력하는 단일 실행 파일(executable)이다:

- 제어 흐름 그래프(control-flow graph): 입력 소스 코드에 대응되는 제어 흐름 그래프
- 오류 정보: 입력 소스에서 발생할 수 있는 오류의 유형과 코드상 위치
- 분석 단계별 상태: 분석을 수행하며 단계적으로 계산된 중간 계산 결과

명령줄 인자를 통해 .go 파일을 입력받으며, 위의 정보는 모두 표준화된 JSON 형식으로 출력된다. 이에 따라 향후 TraceInspector 인터페이스 뿐만 아니라 IDE와의 통합 등도 가능하다.

정적 분석은 "프로그램의 실제 실행 동작을 코드 분석을 통해 추론하는 과정"이다. 그렇다면 실제 실행 동작이 무엇인지, 코드 분석은 어떻게 할 것인지, 그리고 무엇을 추론할지에 대해서도 정의해야 한다. 각 요소들이 어떻게 정의되는지 보다 엄밀한 소개를 하고자 한다.

배경: 정적 프로그램 분석과 결정가능성(decidability)

상술했듯이 정적 프로그램 분석은 실행하지 않고 동작 특성을 추론하는 기법이다. 동작 특성의 경우 다음과 같은 예시들이 존재한다:

- 0으로의 나눗셈을 수행하게끔 하는 입력이 존재하는가?
- 배열 인덱싱은 항상 올바른 범위 내에서 수행되는가?
- null 포인터를 참조하지 않는가?
- 로그인하지 않은 상태에서 사용자 정보에 접근할 수 있는가?



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

- 모든 입력에 대해 프로그램이 유한한 시간 내에 종료하는가?

이처럼 실행했을 때의 행동과 관련되어 있고, 참 또는 거짓으로 판별되는 명제들을 의미론적 성질(semantic property)로 칭하게 된다. "의미론적" 성질인 이유는 단순히 프로그램의 겉모양인 구문(syntax)만 분석해서 판별할 수 없기 때문이다.

코드 1은 콜라츠 수열(Collatz sequence)의 n 번항을 계산하는 함수이며 다음과 같은 정의를 가진다:

$$C(n) = \begin{cases} \frac{n}{2} & n \text{ 은 짝수인 경우} \\ 3n + 1 & n \text{ 은 홀수인 경우} \end{cases}$$

```
def collatz(n):
    if n == 1:
        return 1
    if n % 2 == 0:
        return collatz(n / 2)
    else:
        return collatz(3 * n + 1)
```

코드 1: Collatz 수열을 계산하는 코드

모든 양수 n 에 대해서 위 함수가 종료할까? 1937년도에 수학자 콜라츠는 "그렇다"고 추측했지만, 아직까지도 그가 맞았다는 증명이 존재하지 않는다. 이에 따라 특정 입력값에 대해서는 실행을 통해 확인할 수 있지만, 모든 양수 입력에 대해 종료함을 보장하는 일반적인 방법은 현재까지 알려져 있지 않다.

비슷하게 튜링의 유명한 정지 문제(halting problem)[14]가 있다: 튜링의 정지 문제는 "임의의 프로그램이 주어졌을 때, 해당 프로그램이 유한한 시간 내에 정지할지 정확히 판별하는 프로그램은 존재할 수 없다"는 정리이다.

콜라츠 추측과 정지 문제와 같이 우리는 답을 정확히 알 수 없는 의미론적 성질이 존재함을 알게 되었다. 그렇다면 비자명한 의미론적 성질 중 자동으로 완벽하게 판별 가능한 것이 단 하나라도 존재할까? 놀랍게도 Rice 정리에 의하면 **존재하지 않는다**:

정리 (Rice[11], 1953). L 이 튜링 완성 언어이고, P 가 L 의 프로그램들에 대한 비자명한¹ 의미론적 성질일 때, L 의 모든 프로그램 p 에 대해

$$A(p) = \begin{cases} \text{참} & p \text{ 가 } P \text{ 를 만족하는 경우} \\ \text{거짓} & p \text{ 가 } P \text{ 를 만족하지 않는 경우} \end{cases}$$

를 만족하는 판별 알고리즘 A 는 존재하지 않는다.

¹non-trivial: 일부 프로그램에 대해서는 참이고, 일부 프로그램에 대해서는 거짓인 성질



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

이를 풀어 말하자면 어떤 튜링 완성 언어로 작성된 임의의 프로그램 p 가 성질 P 를 만족하는지 완벽하게 판별 가능한 알고리즘은 존재할 수 없다는 의미이다. 여기서 P 를 "프로그램이 언젠가는 종료한다"로 한정하면 정지 문제가 된다.

논리학과 계산과학에서는 이처럼 어떤 명제에 대해 참 또는 거짓을 판별할 수 있는 알고리즘의 존재 여부를 결정가능성(decidability)[9]이라고 칭한다. 이러한 관점에서 정적 프로그램 분석은 의미론적 성질을 판별하는 문제이기에 결정 불가능(undecidable)하며, 이에 따라 "모든 프로그램"에 대해 "자동으로", "완벽한" 정확성을 가지는 정적 분석기는 구현이 불가능하다.

결국 "모든 프로그램에 대해 동작", "사용자의 개입 없는 알고리즘", "완벽한 정확성(분석 결과가 실제 오류 존재 여부와 동치)" 중 최소한 하나의 조건은 포기해야 분석기 구현이 가능해진다.

배경: 프로그램 의미론

프로그램이 실행되었을 때 동작하는 과정을 어떻게 표현할까? 단순히 생각하면 메모리의 특정한 위치에 특정한 값을 계산한 후 저장한다고 정의할 수 있다. 형식 프로그램 의미론(formal program semantics)은 이러한 프로그램의 동작 과정에 대한 엄밀한 수학적 모형을 제시한다. 본 문서에서는 형식 의미론 중 operational semantics에 대해 소개하고자 한다. Tracelnspector의 요약해석 구현이 operational semantics 위에서 정의되었기 때문이다.

Operational semantics는 프로그램 실행 과정을 프로그램 상태간의 전이로 나타내는 기법이다. 그림 3는 어떤 시작 메모리 상태 S 로부터 프로그램이 실행을 시작해서 $a_1, a_2, \dots, a_n$ 중 하나의 상태로 전이되고, 이후에 후속 상태들로 계속해서 전이하는 구조로 나타낸 예시이다. 마치 오토마타에서 시작 상태에서부터 주어진 규칙과 입력에 의해 다음 상태로 전이되는 것과 같이, 프로그램 의미론 역시 주어진 상태와 입력에서 언어의 문법적 요소별로 정해진 규칙에 따라 다음 상태로 전이되는 체계로 나타낸 형식이다.

요약해석 맥락에서는 이처럼 실제 프로그램 실행을 묘사하는 의미론을 "구체적 의미론(concrete semantics)"이라 칭한다. Tracelnspector에서 정의한 구체적 의미론 전체는 ??에 수록하였다.

이러한 구체적 의미론 위에서 주어진 입력과 초기 상태에 대해 실행 시 거치는 상태들의 연속을 실행 흔적(trace)라 칭한다. $S \hookrightarrow a_1 \hookrightarrow b_1 \hookrightarrow c_1$ 흔적을 예시로 살펴보면, 시작 상태 S 로부터 a_1, b_1 을 거쳐 최종 상태 c_1 에서 종료된 하나의 실행 흔적이다.

오류 상태란 어떤 명제 P 를 만족하는(혹은 만족하지 않는) 상태들의 집합이라 볼 수 있다. 그림 4에선 오류 상태들을 $e_1, \dots, e_7$ 로 정의하였다. 이에 따라 프로그램의 초기 상태에서부터 시작하여 e_n 에 도달 가능한 경로 $S \hookrightarrow a_n \hookrightarrow \dots \hookrightarrow e_n$ 가 존재하는 경우 오류가 발생할 수 있는 프로그램으로 정의할 수 있다.

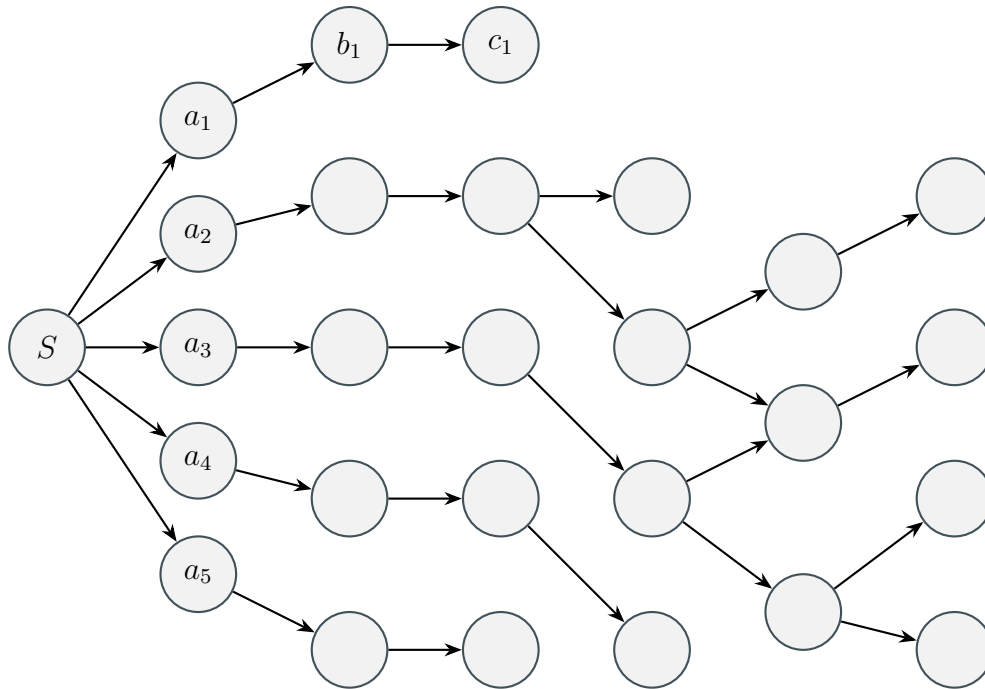


그림 3: 프로그램 상태 간 전이를 통해 의미를 나타내는 operational semantics

배경: 요약해석(abstract interpretation)

이제 프로그램이란 메모리 상태간의 전이이고, 전이 방법은 해당 프로그래밍 언어의 형식 의미론에 의해 정해지며, 오류란 특정한 조건을 만족하는 메모리 상태이며, 시작 상태에서부터 도달 가능한 오류 상태가 존재하는 경우 오류가 존재하는 프로그램이라는 것을 정의하였다.

요약해석 이론은 논리적으로 건전한(sound) 분석을 목표로 한다. 신뢰할 수 있는 분석기를 만든다는 것은 다양한 해석이 가능하겠지만 그중 "오류가 없다고 분석된다면 실제로 실행 가능한 모든 경로에 대 오류 흔적은 존재하지 않는다" 고 정의할 수 있으며, 이 문장이 논리적 건정성을 한 문장으로 표현한 것이다.

입력 프로그램 p 에 대해, $analysis(p)$ 는 p 에 대한 분석을 수행하며, 그 결과로 "오류가 존재한다"와 "오류가 존재하지 않는다"를 반환한다. $error(p)$ 는 p 를 실행했을 때 오류가 발생 가능한지 판별하는 오라클(oracle) 함수이다. 여기서 오라클이란 실제로 구현 불가능하지만 완벽하게 사실을 판별할 수 있는 가상의 함수이다. $analysis(p)$ 가 논리적으로 건전하기 위해선

$$analysis(p) = \text{"오류가 존재하지 않는다"} \Rightarrow error(p) = \text{"오류가 존재하지 않는다"} \quad (1)$$



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

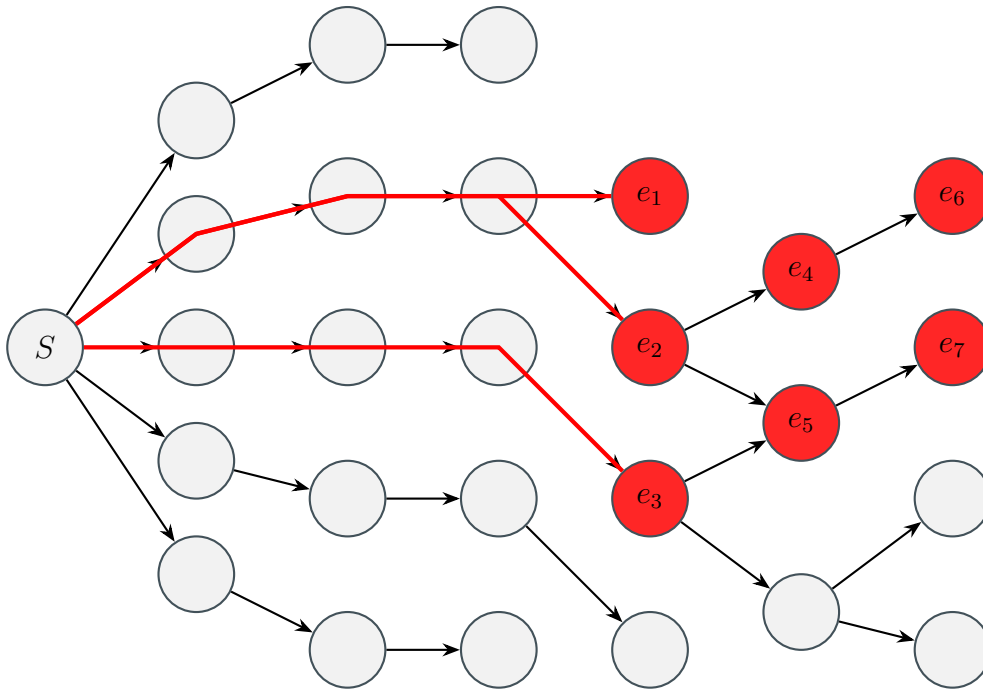


그림 4: 오류 상태 및 흔적

를 만족해야 한다.² 이 조건은 오류가 존재하지 않는 분석 결과에 대해서는 보증을 하지만, 반면 분석 결과가 "오류가 존재한다"인 경우 어떠한 보증도 하지 않게 된다. 실제로 실행했을 때 오류가 발생할수도 있지만 그렇지 않고 존재하지 않는 오류에 대한 허위 정보(false positive)를 낼 수도 있다는 의미이다. 앞서 설명한 Rice 정리의 직접적인 영향 때문이다.

요약해석 이론은 실제 프로그램 상태에 대한 "보수적인" 근사치를 논리적으로 건전하게 계산하는 방법이다. 앞서 활용한 프로그램 흔적 예시를 요약해석의 관점에서 생각해보자면, 그림 5와 같이 모든 상태들에 대한 구체적인 전이를 계산하지 않고, 초록색 상자처럼 여러개의 구체적인 상태들을 요약해서 나타낼 수 있는 "요약 상태"로 압축하여 나타내는 것이다. 오류 상태들에 대해서도 마찬가지로 구체적인 오류 상태들을 포함한 근사치(빨간색 상자)로 오류 상태를 정의하게 된다. 보수적인 근사이기 때문에 요약 오류 상태에서는 실제로는 오류 상태가 아니지만 포함된 일반 상태도 존재하게 된다. 하지만 실제 오류 상태들은 빠짐없이 요약 상태에서도 오류로 표현되어야 한다.

그림 6는 간단한 정수값을 계산하는 프로그램에 대해 구체적인 값들을 계산하는 경우와 각 변수의 부호만을 요약해서 계산하는 경우를 나타내었다. 구체적인 값의 경우 x 가 임의의 입력 $n \in \mathbb{Z}$ 을 받은 이후, 코드에 따라 변수 값을 직접 계산하게 된다. 반면 부호로 요약한 경우 잠재적으로 가능한 입력은 모든 정수이기에, $-$, 0 , $+$ 중 어떤 값도 가능한 $\top$ 기호를 통해 나타나게 되고, 이후 조건에 따라 양수, 음수, 0 으로 나뉘서 표현하게 된다. 부호 정보만을 가지고 있기에 실제로 정수의 값을 알 순 없지만, 실제 값을 알지 못하더라도 각

²더 일반적이고 엄밀한 정의는 임의의 의미론적 명제 P 에 대해 $analysis(p, P) = true \Rightarrow P \ p$ 이다.

프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

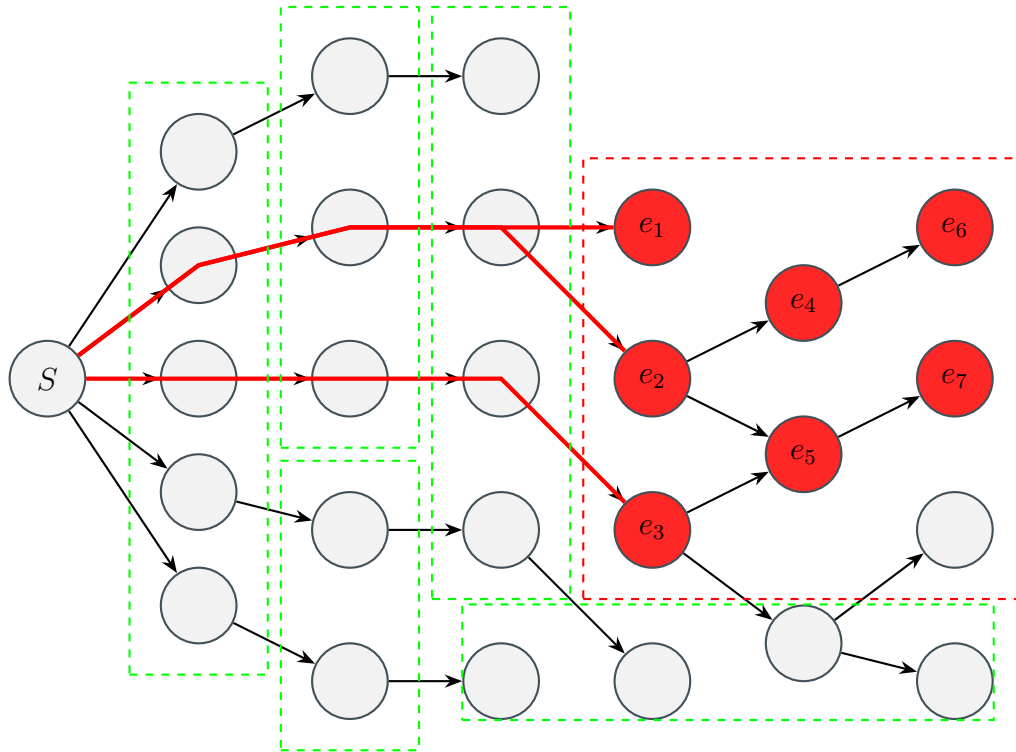


그림 5: 오류 상태의 요약

분기에 들어갔을 때 가질 수 있는 값들에 대한 성질을 추론할 수 있게 된다.

조금 더 엄밀하게 생각해보면 각 코드의 지점마다 변수가 가질 수 있는 값들은 어떤 조건을 만족하는 집합으로 나타낼 수 있다. $x = \text{input}()$ 의 경우 정수집합 $\mathbb{Z}$ 가 가능하며, 양수의 경우 양수들의 집합 $\mathbb{Z}_+ := \{n \mid n > 0\}$, 음수는 $\mathbb{Z}_- := \{n \mid n < 0\}$, 그리고 한원소 집합 $\{0\}$ 로 표현된다. 따라서 정수값들을 다루는 의미에서 프로그램 상태는 정수집합 $\mathbb{Z}$ 의 멱집합 $\mathcal{P}(\mathbb{Z})$ 가 된다.

부호들의 경우 보다 단순하다. "모든 부호"를 나타내는 $\top$ 와, $+$, 0 , $-$ 로 구성되어 있다. 추가로 위 코드에서는 쓰이지 않았지만, "모든 부호"의 쌍대에 해당하는 "유효하지 않은 값들의 부호"를 나타내는 $\perp$ 도 추가된다. 즉 부호로 요약해서 계산하는 의미론에서는 프로그램 상태가 부호 요약도메인 $\text{sign} := \{\top, +, 0, -, \perp\}$ 의 원소들로 표현된다.

따라서 부호를 계산하는 것이란 주어진 코드 지점에서 구체적인 정수값들의 집합 $s \subset \mathbb{Z}$ 에서 sign 으로 대응시키는 함수 $\alpha : \mathcal{P}(\mathbb{Z}) \rightarrow \text{sign}$ 으로 볼 수 있다. $x = \text{input}()$ 를 수행한 이후 x 가 가능한 값들의 집합은 $\mathbb{Z}$ 이고 이에 따른 $\alpha(\mathbb{Z}) = \top$ 가 되고, 마찬가지로 x 가 양수인 집합은 $\mathbb{Z}_+$, $\alpha(\mathbb{Z}_+) = +$ 가 된다.

$x := 4$ 와 같은 코드가 실행된 이후 가능한 상태의 집합은 한원소 집합 $\{4\}$ 이다. $\alpha(\{4\}) = +$ 가 되는데, 이는 $\alpha(\mathbb{Z}_+)$ 와 동일한 결과를 가지게 된다. 양수들의 집합보다 구체적인 정보를 가지고 있었지만 부호로 요약되며 해당 정보를 잃게 되는 것이다. 그림 6는 정수 집합을 부호 요약도메인으로 대응시키는 예시를 보여준다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

```
x = input() [x : n]
if x < 0:
    y = -x [x : n, y : -n]
elif x > 0:
    y = x [x : n, y : n]
else:
    y = 0 [x : n, y : n]
```

(a) 구체적 정수값

```
x = input() [x : T]
if x < 0:
    y = -x [x : -, y : +]
elif x > 0:
    y = x [x : +, y : +]
else:
    y = 0 [x : 0, y : 0]
```

(b) 부호 도메인으로 요약

그림 6: 정수값과 부호 도메인

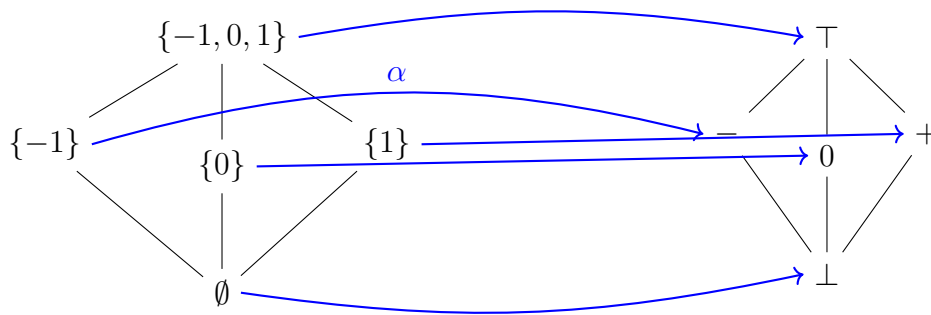


그림 7: 정수 집합과 부호로의 대응

구체적인 값들의 집합에서 요약도메인의 원소로 대응시키는 α 함수가 있듯이, 역으로 요약도메인의 원소에서 구체적인 값들의 집합으로 대응시키는 $\gamma : \text{sign} \rightarrow \mathcal{P}(\mathbb{Z})$ 를 정의할 수 있다. $\gamma(\top) = \mathbb{Z}, \gamma(+)=\mathbb{Z}_+, \dots$ 로 간단하게 정의된다.

두 함수 α, γ 에 대해서 다음과 같은 성질에 대해 생각해볼 수 있다:

- 임의의 정수집합 $s \subseteq \mathbb{Z}$ 에 대해, $\gamma(\alpha(s))$ 의 결과
- 임의의 부호도메인의 원소 $e \in \text{sign}$ 에 대해, $\alpha(\gamma(e))$ 의 결과

첫번째 경우는 $\alpha(s)$ 를 수행하며 구체적인 정보가 소실된다. 이에 따라서 $\{4\} \subset \mathbb{Z}_+$ 와 같이 보다 구체적인 집합을 넣더라도 같은 도메인 값 $\alpha(s)$ 으로 요약된다. $\gamma(e)$ 는 해당 부호를 만족하는 원소들을 전부 포함시키기에, 모든 $s \subseteq \mathbb{Z}$ 에 대해 $s \subseteq \gamma(\alpha(s))$ 임을 알 수 있다.[4]

$\gamma(e)$ 의 결과는

- $\gamma(\top) = \mathbb{Z}$
- $\gamma(\perp) = \emptyset$
- $\gamma(+)=\mathbb{Z}_+$



- $\gamma(0_{\text{sign}}) = \{0\}$
- $\gamma(-) = \mathbb{Z}_-$

로 열거할 수 있다. 따라서 모든 $e \in \text{sign}$ 에 대해 $e = \alpha(\gamma(e))$ 임을 알 수 있다.

α, γ 두 함수에 대한 이 성질들은 "보수적인" 요약이 무엇인지를 정확하게 나타내는 역할을 한다. 구체적인 값들의 집합을 요약하여 표현한 결과는 항상 원래 값을 포함하여야 함을 정의한 것이고, 이 조건을 통해 요약 상태에서 추론을 하더라도 구체적인 상태를 빠짐없이 반영할 수 있음을 보장하게 된다.

지금까지 설명은 정수들의 집합 $\mathcal{P}(\mathbb{Z})$ 에 대한 순서관계 $\subseteq$ 가 정의된 상태에서 진행되었는데 마찬가지로 요약도메인 sign 에 대해서도 순서관계 $\sqsubseteq$ 를 정의할 수 있다: $\perp$ 는 어떠한 값도 나타내지 않기에 $\forall e. \perp \sqsubseteq e$ 가 되며, 마찬가지로 $\top$ 는 모든 정수값을 나타내기에 $\forall e. e \sqsubseteq \top$ 가 성립된다. $+, 0, -$ 는 구체적인 상태로도 서로소 집합이기에 포함관계로도 순서 정의가 불가능하다. 이는 "부분 순서 관계(partially ordered)"로 정의가 되며, 모든 원소보다 순서 관계상 큰 $\top$ 원소와 작은 원소 $\perp$ 가 존재하기에 "완전 부분 순서 관계(complete partial order)"의 성질을 가지게 된다.

따라서 위에서 정의한 α, γ 의 두 성질은 순서 관계가 정의된 두 집합 $(\mathcal{P}(\mathbb{Z}), \subseteq)$ 와 $(\text{sign}, \sqsubseteq)$ 사이의 "갈루아 연결(galois connection)"을 나타낸다:

정의 (갈루아 연결(Galois Connection)). 프로그램의 구체적 상태들의 집합 C 와 그 위의 순서 관계 $\subseteq$ 가 주어지고 요약상태 집합 A 와 그 위의 순서 관계 $\sqsubseteq$ 가 주어졌을 때, 두 함수 $\gamma: A \rightarrow C$ 와 $\alpha: C \rightarrow A$ 에 대해

$$\forall c \in C. \forall a \in A. \alpha(c) \sqsubseteq a \iff c \subseteq \gamma(a)$$

를 만족하면 두 집합과 순서관계 사이에 갈루아 연결을 형성한다고 정의하며,

$$(C, \subseteq) \overset{\alpha}{\underset{\gamma}{\rightleftarrows}} (A, \sqsubseteq)$$

로 나타낸다.[3][12]

이는 단순히 "정수와 부호"와 같이 특정한 값들과 요약 상태뿐만 아니라 일반적으로 적용할 수 있는 성질이며, 따라서 요약해석에 기반한 정적분석을 구현할 때 논리적으로 건전함을 보이기 위해선 프로그램의 구체적 상태와 요약 상태간의 갈루아 연결이 형성됨을 증명해야 한다.

TraceInspector는 algebra/extended_integers.go에 $\pm\infty$ 를 추가한 확장된 정수체계 $\mathbb{Z}_\infty$ 를 구현하였다. 대소관계 비교 및 산술연산이 구현되어 있으며, 이를 기반으로 domain/interval.go에 확장된 정수체계 기반 구간도메인이 구현되어 있다.

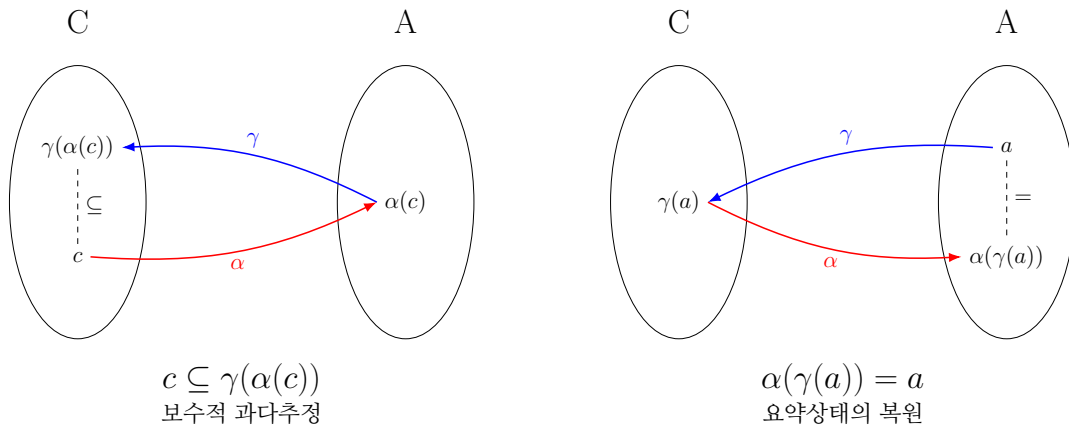


그림 8: 갈루아 연결과 α, γ 의 성질

Imp 중간 표현 언어

TraceInspector의 첫 단계는 입력으로 Go 소스 파일을 받아 Go 표준 라이브러리의 go/parser 패키지를 이용해 추상 구문 트리(abstract syntax tree, AST)로 파싱하는 것이다. 이후 Imp[10]라는 간단한 명령형 프로그래밍 언어 겸 중간 표현 형식으로 번역한 후 ImpAST위에서 분석 단계를 수행한다.

Imp는 다음과 같은 기능을 가진 명령형 언어이다:

- int, bool, string 및 다차원 동적 배열 []T
- 산술/논리 연산 및 입출력
- if, while, continue, break 제어 흐름
- 함수 선언 및 반환

이처럼 구조적으로 간단하지만 충분히 복잡한 수준의 프로그램을 표현할 수 있는 구조이다.

Go AST에 대해 직접적으로 분석을 수행하지 않는 이유는 크게 두가지가 있는데, 첫째는 추상 구문 트리의 특성상 프로그램의 제어 흐름을 제외한 구문적 요소들이 트리에 많이 포함되어 있기 때문이다. 그림 9는 b라는 변수에 원소가 100인 정수형 동적 배열을 선언하는 명령 `b := []int{100}`를 Go AST로 파싱한 결과와, 이를 ImpAST로 번역한 결과를 나타낸다. Go AST의 경우 리터럴에 대해서도 종류와 값을 분리하여 표기하고, 배열 리터럴 역시 타입을 구별하여 표기하는 등 범용적이지만 복잡한 형태로 구성되어 있다.

반면 Imp의 경우 리터럴의 타입별 정의와 배열 리터럴 등 문법적 요소를 최대한 배제한 형태로 구현하였고, 리터럴 값을 문자열로 표현한 것이 아니라 타입에 맞는 값을 직접 담게 된다. 이러한 설계는 향후 제어 흐름 그래프 생성 등 프로그램의 형태가 아닌 동작 특성을 파악하고자 하는 경우에 훨씬 간결하고 수월하게 구현을 가능케 해준다.

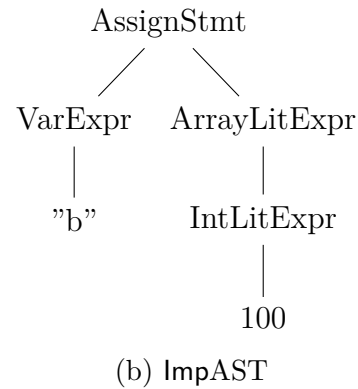
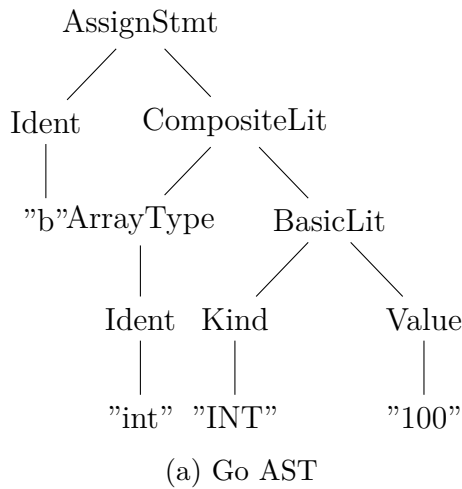


그림 9: Go AST와 ImpAST 비교

두번째 이유는 특정 언어의 문법적 형태와는 독립적인 분석 기능을 구현하고자 했기 때문이다. Go 언어의 요소에 대한 요약의미론을 직접 정의할 경우 오직 Go 언어에 대해서만 분석 가능하지만, Imp라는 전형적인 언어를 대상으로 분석기를 구현하고자 할 경우, 향후 Go 외의 타 언어를 Imp로 번역하는 번역기만 구현할 경우 이후 분석기 로직을 온전히 활용할 수 있다.

이처럼 입력 소스를 파싱하고 처리하는 프론트엔드와, 분석을 담당하는 미들엔드를 분리하여 각각 다른 표현상에서 동작하게끔 설계하는 방식은 현대 컴파일러에서 자주 활용되는 매우 전형적인 설계 방식이다. TraceInspector 역시 이러한 설계 패턴을 도입하고, 구현의 편의성을 위해 Imp로의 번역 과정을 최우선적으로 수행한다.

Imp의 문법적 정의는 그림 10과 같으며, Imp의 bigstep operational semantics 일부는 부록 5.4에 수록하였다.

요약 도메인

TraceInspector는 정수값을 구간 요약도메인으로, bool값을 bool 요약도메인으로, 그리고 배열을 배열 축약 요약도메인으로 요약하여 실행한다.

구간 요약도메인 $\mathbb{I}$ 란 가능한 정수들의 집합을 $[l, h]$ 의 닫힌 구간으로 나타낸 요약 도메인이다.

구간 요약도메인에 대한 α, γ 정의는 다음과 같다:

- $\alpha(\mathbb{Z}) = [-\infty, \infty] = \top$
- $\alpha(\emptyset) = \perp$
- $\alpha(\{x_0, x_1, \dots, x_n\}) = [\min(x_0, x_1, \dots, x_n), \max(x_0, x_1, \dots, x_n)]$



```

 $e ::= \mathbb{X} \mid \mathbb{N} \mid \mathbb{B} \mid \mathbb{S} \mid \{e_1, \dots, e_n\}$ 
|  $e + e \mid e - e \mid e * e \mid e / e \mid e \bmod e$ 
|  $e = e \mid e \neq e \mid e < e \mid e \leq e \mid e > e \mid e \geq e$ 
|  $e \wedge e \mid e \vee e$ 
|  $\neg e \mid \neg e$ 
|  $(e) \mid e[e]$ 
|  $f(e_1, \dots, e_n)$ 
|  $\text{len}(e)$ 
|  $\text{make\_array}(e, e)$ 

 $s ::= \text{skip}$ 
|  $x := e$ 
|  $e[e] := e$ 
| if  $e$  then  $\{s^*\}$  else  $\{s^*\}$ 
| while  $e$  do  $\{s^*\}$ 
| break | continue
|  $e++ \mid e--$ 
|  $f(e_1, \dots, e_n)$ 
| print $(e_1, \dots, e_n)$ 
| scanf $(s, e_1, \dots, e_n)$ 
| return  $e$ 

 $\tau ::= \text{int} \mid \text{bool} \mid \text{string}$ 
|  $\text{array}\langle \tau \rangle$ 

```

그림 10: Imp의 명령문(statement), 식(expression)들의 문법

- $\gamma(\top) = \mathbb{Z}$
- $\gamma(\perp) = \emptyset$
- $\gamma([l, h]) = \{i \in \mathbb{Z} \mid l \leq i \leq h\}$

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

구간끼리의 산술은 다음과 같다:

$$[l_1, u_1] + [l_2, u_2] = [l_1 + l_2, u_1 + u_2]$$

$$[l_1, u_1] \times [l_2, u_2] = [\min(l_1u_1, l_1u_2, l_2u_1, l_2u_2), \max(l_1u_1, l_1u_2, l_2u_1, l_2u_2)]$$

$$-[l, u] = [-u, -l]$$

$$i_1 \sqcup i_2 = \begin{cases} i_2, & \text{if } i_1 = \perp \\ i_1, & \text{if } i_2 = \perp \\ [\min(l_1, l_2), \max(u_1, u_2)] & \text{where } i_1 = [l_1, u_1], i_2 = [l_2, u_2] \end{cases}$$

갈루아 연결을 형성하기 위해 구간도메인 $i_1, i_2 \in \mathbb{I}$ 간의 순서관계 $i_1 \sqsubseteq i_2$ 를 다음과 같이 정의한다:

$$i_1 \sqsubseteq i_2 = \begin{cases} \text{true}, & \text{if } i_1 = \perp \\ i_1 = \perp, & \text{if } i_2 = \perp \\ l_2 \leq l_1 \wedge u_1 \leq u_2, & \text{where } i_1 = [l_1, u_1], i_2 = [l_2, u_2] \end{cases}$$

죽 i_1 이 i_2 의 범위 내에 완전히 포함되어 있는 경우 $i_1 \sqsubseteq i_2$ 가 성립한다.

그림 11은 정수 요약도메인을 격자로 표현한 그림이다³. 이렇게 정의된 정수 구간도메인은 그 격자의 높이가 무한히 높다. 따라서 widening 기법을 적용하여 무한히 증가하는 join을 강제로 종료시켜 주기 위한 widening 연산 ∇ 이 필요하다:

$$[n, p] \nabla [n, q] = \begin{cases} [n, p], & \text{if } p \geq q \\ [n, +\infty], & \text{if } p < q \end{cases}$$

Widening 연산은 일정 횟수 이상 상한(혹은 하한)이 단조 증가(감소)하는 경우 정확도를 희생하여 모든 정수들의 상계(하계)인 $\pm\infty$ 를 치환하여 분석이 종료할 수 있도록 유도하는 역할을 한다.

Boolean 값은 **true**, **false** 만 존재하기에 요약도메인을 그림 12과 같이 정의할 수 있다. $\top$ 의 경우 **true**, **false** 가 모두 가능한 경우이며, $\perp$ 는 유효한 boolean 값 산출이 불가능한 경우이다.

³Patrick Cousot: Principle of Abstract Interpretation의 33.1장에서 발췌함

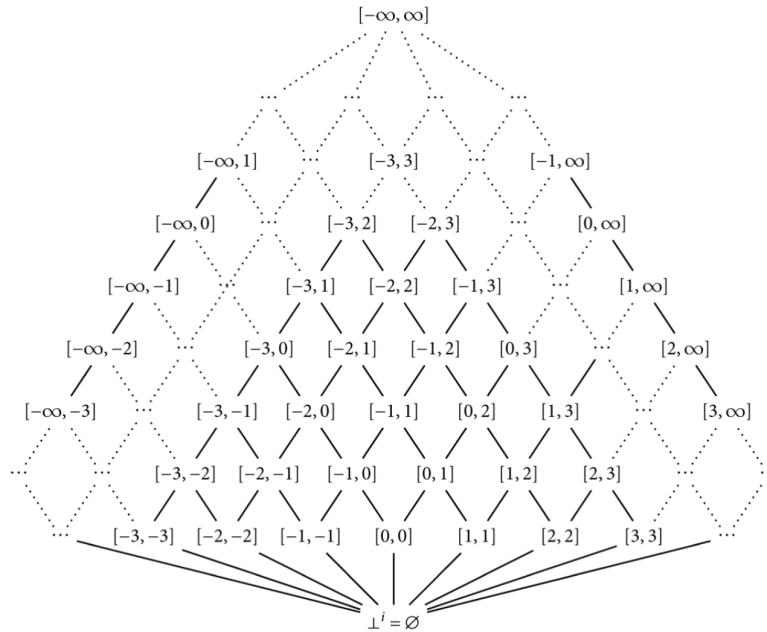


그림 11: 확장된 정수 구간도메인 $\mathbb{I}_{\mathbb{Z}_{\infty}}$

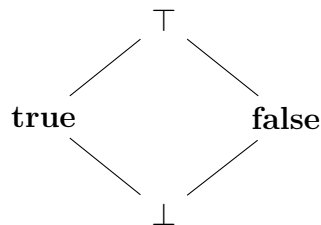


그림 12: Boolean 요약도메인

제어 흐름 그래프(control-flow graph)

간략화된 AST가 생성된 이후 TraceInspector는 if, while등의 명령문에 의해 프로그램의 실행 흐름을 구조화하기 위한 제어 흐름 그래프(control-flow graph, CFG)구조[1]로 번역한다. 함수별 AST에서 CFG를 생성하는 알고리즘은 다음과 같이 동작한다:

1. 함수의 진입 명령문으로부터 깊이 우선 탐색을 통해 함수의 종말 명령문부터 노드로 번역하여 ID를 부여한다.
2. 후속 노드의 DFS가 종료되면서 현재 노드와 이어줄 간선을 생성한다. 만약 현재 노드가 조건문, 반복문과 같이 분기를 포함하는 경우 각 참, 거짓 명령문별 노드와 간선을 생성해준다.
3. break, continue는 현재 동작하고 있는 가장 깊은 반복문에 대해 동작한다. 이를 처리하기 위해서 현재 처리중인 반복문을 스택 구조로 저장하여 중첩된 반복문에 대한

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

```

fun main() none {
  a = 5
  arr = make_array(a, 1337)
  i = 0
  while i < 10 {
    arr[i] = i
    i = i + 1
  }
  print(arr)
}

```

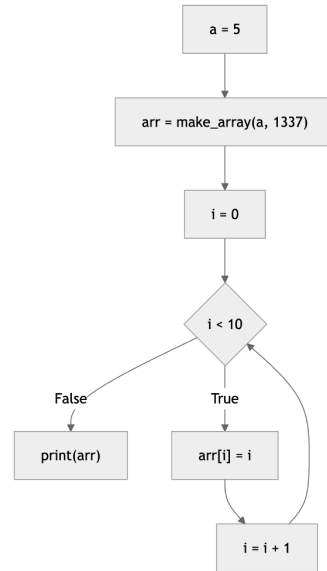
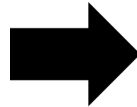


그림 13: 입력 Imp코드와 CFG로의 표현

정보를 저장한다.

4. 이에 따라 `break`, `continue` 및 루프 탈출은 반복문 스택의 top에 대해 동작하며, 반복문을 처리 완료했다면 스택에서 반복문 정보를 제거한다.

그림 13는 예시 Imp코드에 대한 CFG 표현 결과를 나타낸다. 각각의 명령문들이 단일 노드로 표현되며, `while i < 10 {...}` 반복문을 `i < 10` 조건을 하나의 단일 노드로 표현하고 판정 결과에 따라 후속 명령문들을 실행하는 형태로 나타낸 것을 확인할 수 있다.

이러한 CFG구조는 프로그램의 실행 흐름을 직접적으로 노출시키며, `if`, `while`, `break`, `continue` 등의 문법적인 경우를 전부 열거할 필요 없이 조건 노드와 참, 거짓 후속 노드에 대해서만 논할 수 있도록 분석을 간략하게 해준다. 따라서 이후에 TraceInspector분석기는 제어 흐름 그래프 위에서 직접적으로 동작하는 형태로 설계할 수 있으며, Imp언어의 문법적 디테일로부터 보다 자유롭게 추상화된 형태로 동작이 가능하다.

요약실행 의미론

이제 입력 프로그램을 CFG 형태로 표현하여 분석이 수월한 형태의 자료구조로 번역하였다. 또한 Imp의 구체적인 값 타입들을 전부 각각의 요약도메인으로 대응시키는 α 함수도 정의하였다.

Imp의 구체적인 실행의미론에선 프로그램 상태 $\sigma : \mathbb{V} \rightarrow v$ 는 각 변수를 값으로 대응시켰다(부록 참조). 또한 명령문 S 를 프로그램 상태 σ 에서 실행했을 때는 결과 상태 σ' 와 제어 흐름 결과 r 를 반환하며 $\langle S, \sigma \rangle \Downarrow \langle \sigma', r \rangle$ 로 표기했다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

Imp의 실제 실행이 구체적인 값 $\mathbb{V}$ 에 대해 계산을 하는 의미론에 바탕을 두었다면, 분석을 위해선 값들의 요약도메인 A 위에서 실행되는 요약실행 의미론을 정의해야 한다.

요약실행 의미론에서 요약된 값은 $\text{AbstractValue} : \tau \rightarrow v_\tau^\#$ 로 표현된다. 즉 타입 τ 를 해당 타입의 요약도메인 상에서의 값 $v_\tau^\#$ 로 대응하는 구조가 일차적으로 정의된다.

AbstractValue 는 각 변수마다 존재하기에, 주어진 프로그램 상태에서 각 변수 $v \in \mathbb{V}$ 를 각각의 AbstractValue 로 대응하는 $\text{AbstractVarMemMap} : \mathbb{V} \rightarrow \text{AbstractValue}$ 로 명령문을 실행하는 시점에서의 변수들의 요약 상태를 나타낸다.

이후 CFG 노드별 ID $l \in \mathbb{L}$ 를 AbstractVarMemMap 로 대응시키는 함수 단위의 대응 $\text{AbstractNodeMemMap} : \mathbb{L} \rightarrow \text{AbstractVarMemMap}$ 가 정의된다. 여기서 AbstractValue 는 노드 l 에서 "해당 명령을 실행하기 전의 프로그램 상태", 즉 precondition 상태를 나타낸다.

최종적으로 프로그램 전체의 요약 메모리 상태는 입력 프로그램의 함수들 $f \in F$ 에서 함수별 요약 메모리 상태 $\text{AbstractNodeMemMap}_f$ 로 대응하는 $\text{AbstractFunctionMem} : F \rightarrow m^\#$ 로 나타낸다.

요약실행 의미론은 CFG 노드 l 을 요약 메모리 상태 $M^\# : \text{AbstractVarMemMap}$ 에서 실행했을 때 후속으로 실행할 CFG 노드 l' 과 갱신된 요약 메모리 상태 $M'^\# : \text{AbstractVarMemMap}$ 를 반환하는 단일 실행 단계로 표현할 수 있으며, $(l, M^\#) \hookrightarrow^\# (l', M'^\#)$ 로 표기한다.

이후 l' 에 대한 갱신된 요약 프로그램 상태 $M'^\#$ 과 기존에 계산된 요약 프로그램 상태 $\text{AbstractNodeMemMap}(l')$ 과의 join연산 $M'^\# \sqcup \text{AbstractNodeMemMap}(l')$ 을 수행하여 l' 의 상태를 갱신한다.

따라서 요약 실행 알고리즘은 다음과 같이 설명할 수 있다:

1. 주어진 함수 f 에 대한 초기 상태 $S : \text{AbstractVarMemMap}$ 와 진입 노드 l_{init} 가 주어졌을 때 $(l_{\text{init}}, S) \hookrightarrow^\# (l', M'^\#)$ 을 계산
2. 기존의 $\text{AbstractNodeMemMap}(l')$ 를 $\text{AbstractNodeMemMap}(l') \sqcup M'^\#$ 로 갱신
3. 이후 l' 에서 바로 도달 가능한 후속 노드들 $\text{next}(l')$ 에 대해 위 과정을 반복해서 수행

위 알고리즘은 초기 프로그램 상태에서부터 각 노드에 대해 요약실행 의미론을 적용하여 노드별 상태를 계속해서 갱신하고, 더 이상 변화가 생기지 않는 **고정점**에 도달할 때 까지 반복 수행한다. 이렇게 계산된 고정점 프로그램 상태는 각 노드에서 관측 가능한 모든 값에 대한 정보가 반영된 프로그램 상태이며, 실제로 실행했을 때의 모든 값을 포함한 보수적인 근사 상태가 된다. 컴파일러가 data-flow analysis를 수행할 때 활용하는 Kildall 알고리즘[5] 처럼 CFG위에서 노드별 상태를 계산하는 과정이 유사하다.

그림 14은 요약 실행의미론과 구체적 실행의미론의 관계를 묘사한다. 앞서 설명한 보수적인 근사가 성립하기 위해선, 요약 메모리 상태 $M^\#$ 가 모든 명령문에 대해 $M'^\#$ 로 요약실행이 된다면, $M^\#$ 의 구체적 상태 $\sigma = \gamma(M^\#)$ 에 대해 실제로 실행했을 때 얻는 상태 σ' 는 $M'^\#$ 의 구체적 상태 $\gamma(M'^\#)$ 포함되어 있어야 한다.⁴

⁴보다 엄밀한 정의는 메모리 상태 $\mathbb{V}$ 의 각 변수에 v 에 대한 pointwise-lifted 명세로 정의한다: $\forall v \in \mathbb{V}. \sigma[v] \in$

 국민대학교 소프트웨어학부 다학제간캡스톤디자인 I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

$$\begin{array}{ccc}
 M^\# : \text{AbstractVarMap} & \xrightarrow{\hookrightarrow^\#} & M^{\#'} : \text{AbstractVarMap} \\
 \downarrow \gamma & & \downarrow \gamma \\
 \sigma : \mathbb{V} \rightarrow v & \xrightarrow{\hookrightarrow} & \sigma' : \mathbb{V} \rightarrow v
 \end{array}$$

그림 14: 요약 실행단계 $\hookrightarrow^\#$ 와 구체적 실행단계 $\hookrightarrow$ 과의 대응

위 과정을 그대로 실행할 경우 잠재적으로 끝나지 않는 프로그램(무한루프가 포함된 프로그램 등)의 경우 고정점에 도달할 수 없기에 분석도 종료되지 않는다. 이는 $M^\#$ 에서 사용되는 요약도메인의 높이가 무한한 경우 새로 갱신되는 $\text{AbstractNodeMemMap}(l') \sqcup M^{\#'}$ 가 계속해서 증가하기 때문이다. 따라서 무한히 증가하는 요약상태에 대한 고정점을 제공하는 widening 연산 $\text{AbstractNodeMemMap}(l') \nabla M^{\#'}$ 을 적용하여 고정점에 도달할 수 있도록 유도해야 한다. TraceInspector에서 사용한 요약 도메인 중 정수 구간 도메인과 배열 배열 축약 도메인은 그 높이가 무한하기에, 위에서 정의한 widening을 적용하여 분석이 유한한 시간 내에 종료하도록 보장하였다.

최종적으로 TraceInspector에 대한 논리적 건전성을 다음과 같이 정의할 수 있다:

정리 (TraceInspector분석기의 논리적 건전성). *Imp*의 모든 프로그램 s 와 초기 프로그램 상태 σ 에 대해, $M^{\#'} = \text{analyze}(s, \sigma)$ 를 TraceInspector분석기가 계산한 최종 요약 고정점이라 하자. 이때 s 를 σ 에서 실행했을 때 도달 가능한 모든 구체적 프로그램 상태 σ' 에 대해

$$\sigma' \in \gamma(M^{\#'})$$

가 성립한다.

증명은 각 요약도메인과 타입에 대한 갈루아 연결을 보인 후 *Imp*의 명령문에 대한 구조적 귀납으로 실시한다.

이 증명은 TraceInspector의 분석 결과가 주어진 요약 의미론과 요약 도메인이 정의된 구체적 실행의미론에 대한 정확한 근사를 항상 취한다는 것을 보장한다.

오류 상태 판별

계산된 요약 상태 위에서 TraceInspector분석기는 두가지 오류 조건을 검사한다. 첫번째는 배열 인덱싱 $a[i]$ 가 주어졌을 때 가능한 모든 i 의 값들이 a 의 잠재적인 길이보다 작은지, 그리고 0보다 큰지 검사한다. 해당 검사는 요약 도메인상에서 실시하며 보수적인

$$\gamma(M^\#[v])$$

 국민대학교 소프트웨어학부 다학제간캡스톤디자인I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-13

추정이기 때문에 올바른 인덱싱임을 판정하면 모든 a 와 i 의 값에 대해 안전하지만, 반대로 오류가 나지 않을 $a[i]$ 에 대해서도 오류가 발생할 수 있음을 알리게 된다.

두번째 오류 상태는 CFG의 조건 노드에 대해 수행하며, 조건식의 요약도메인 계산값이 $\perp$ 이 아닌 true, false, $\perp$ 인 경우 경고를 한다. 조건식의 요약 결과가 $\perp$ 인 경우 특정 입력들에 대해서는 참 혹은 거짓임을 나타내기에 즉각적으로 조건식의 결과에 대해 알 수 없게 된다. 하지만 요약 고정점 상태에서 결과가 참 또는 거짓인 경우 해당 조건식에 도달했을 때 가능한 모든 값에 대해 참 또는 거짓으로 계산됨을 나타내므로 조건식 자체가 항진명제 혹은 모순명제가 되어 특정 분기가 도달 불가능함을 함의한다.

2.2.1.2 TracelInspector 프론트엔드

2.2.2 시스템 기능 요구사항

2.2.3 시스템 비기능(품질) 요구사항

2.2.4 시스템 구조 및 설계도

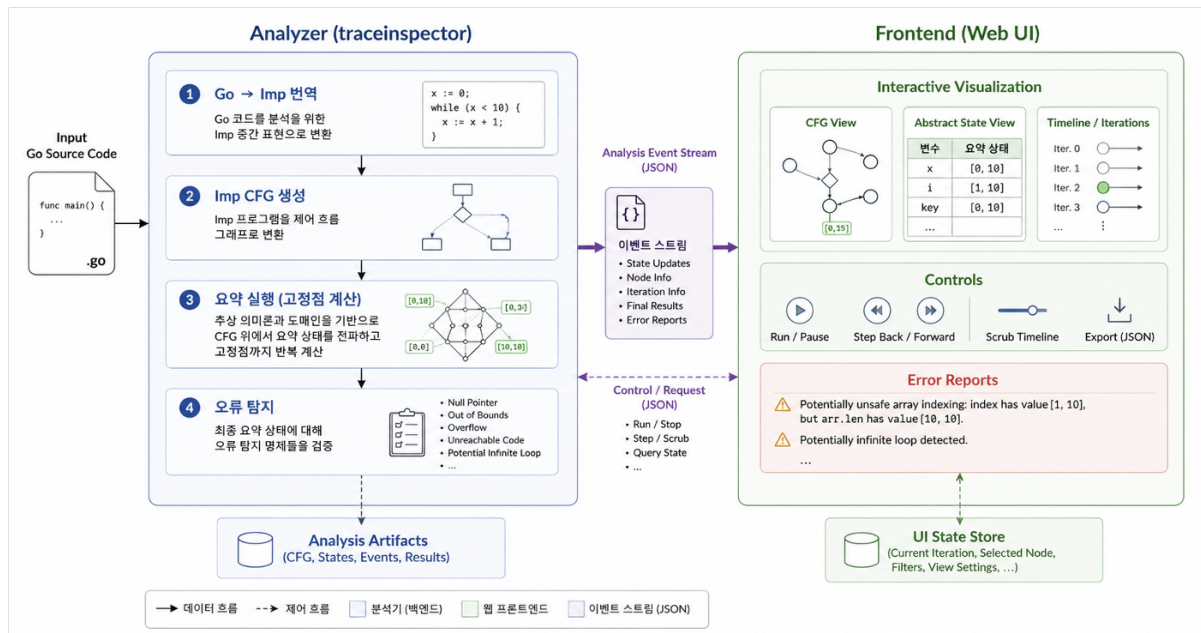


그림 15: TracelInspector전체 시스템 구조

그림 15는 TracelInspector의 전반적인 구조를 나타낸다. TracelInspector분석기는 입력 Go 프로그램에 대한 요약 프로그램 상태를 계산하여 오류 정보와 함께 JSON 형태로 표준 출력에 출력한다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-13

TraceInspector프론트엔드는 사용자로부터 코드를 입력받아 분석기를 호출하여 수행 결과를 시각화하는 웹 인터페이스와 백엔드로 구성되어 있다.

2.2.5 활용/개발된 기술

TraceInspector 분석기는 Go 소스 파일 파싱을 위한 go/parser[13] 패키지를 활용한다. 이후의 모든 분석 단계는 외부 라이브러리를 활용하지 않고 직접 구현되었지만 상술했듯이 요약해석 이론 자체는 잘 알려진 정적 분석 기법이다.

2.2.6 현실적 제한 요소 및 그 해결 방안

TraceInspector분석기는 본질적으로 입력 언어인 Go의 일부분에 대해서만 분석을 수행할 수 있다. 우선적으로 포인터 자료형에 대한 분석을 지원하지 않는다. 요약 실행의미론과 요약 프로그램 상태에서는 포인터나 주소 등의 위치와 참조의 개념 없이 오직 단일 값으로만 간주하였고, 지금의 요약 프로그램 상태는 포인터 자료형에 대한 추론을 할 수 없다. 포인터 자료형에 대한 분석은 abstract heap라는, $\text{AbstractVarMemMap} : \mathbb{V} \rightarrow \text{AbstractValue}$ 가 아닌 주소 $\mathbb{A}$ 를 값으로 대응하는 $\text{AbstractHeap} : \mathbb{A} \rightarrow \text{AbstractValue}$ 를 구현하여 논해야 한다.

2.2.7 결과물 목록

TraceInspector의 소스는 루트 경로 내에 traceinspector 와 front_src 로 크게 나누어져 있다.

traceinspector 경로는 분석기 구현체를 담고 있으며, 다음과 같은 하위 경로로 세분화되어 있다.

algebra 경로에는 분석기에서 활용되는 확장된 정수체계 및 대수적 논리식 조작 체계를 수록하고 있다:

- extended_integers.go는 무한대로 확장한 정수체계 $\mathbb{Z}_\infty$ 를 나타내는 ExtInt 구조체와 관련 연산들을 구현한다.
- simple_prop.go는 $\pm x \pm y \odot C$ 형태의 대수식을 나타내는 SimpleProp 구조체와 수식 조작, 그리고 $\text{SimpleProp} \Leftrightarrow \text{Imp식}(\text{expression})$ 간의 변환을 구현한다.
- arith_normalizer.go는 $a_1x_1 + a_2x_2 + \dots + a_nx_n + C$ 형태의 다항식을 나타내는 Polynomial 구조체와 수식 조작, 그리고 마찬가지로 Imp식간의 변환을 구현한다.

cmd 경로에는 분석기의 실행파일 생성을 위한 진입점 및 명령줄 인터페이스를 구현한 main.go를 수록하고 있다.

domain 경로에는 TraceInspector분석기의 요약 도메인 추상 인터페이스 및 정수 구간과 Bool 요약도메인, 그리고 filter 함수 $\mathcal{F}_B$ 인터페이스 및 구현을 수록하고 있다:



- domain.go는 요약도메인의 추상 인터페이스 AbstractDomain와 정수 요약도메인의 추상 인터페이스 IntegerDomain를 구현한다.
- array_top_domain.go는 모든 배열을 T로 요약하는 약식 배열 요약도메인을 구현한다.
- bool_domain.go는 Bool 요약도메인 및 관련 연산을 구현한다.
- interval.go는 구간 요약도메인 및 관련 연산을 구현한다.
- filter_query.go는 filter 함수 구현을 위한 FilterQuery 구조체 및 논리식의 형태를 정의한다.
- filter.go는 SimpleProp로부터 FilterQuery를 구할 수 있는 변환 함수 및 대수적 조작 함수들을 구현한다.

imp 경로에는 Imp의 정의와 Go → Imp번역기, 그리고 Imp의 구체적 실행의미론을 구현한 인터프리터를 수록하고 있다.

- imp.go는 Imp의 문법과 타입 정의, AST 구조체들을 구현한다.
- interpreter.go는 Imp의 구체적 실행의미론을 구현한, Imp프로그램을 실행하는 인터프리터를 구현한다.
- go2imp.go는 Go AST를 ImpAST로 번역하는 번역기를 구현한다.

탐레벨 traceinspector 경로에는 다음과 같은 중요 소스 파일들이 존재한다:

- abstract_mem.go는 AbstractValue, AbstractVarMemMap, AbstractNodeMemMap, 그리고 함수별 요약상태 저장용 AbstractFunctionMem 을 구현한다.
- analyzer.go는 고정점 계산 및 Imp의 요약 실행의미론을 구현한 AbstractAnalyzer 구조체를 구현한다.
- array_domain.go는 배열 타입에 대한 요약도메인의 추상 인터페이스 ArrayDomain를 구현한다.
- array_summary_domain.go는 배열 원소 축약 요약도메인을 구현한다.
- cfgdefs.go는 CFG 노드 및 간선들과 관련된 정의를 구현한다.
- create_cfg.go는 ImpAST를 CFG로 번역하는 알고리즘을 구현한다.



2.3 기대효과 및 활용방안

3 자기평가

4 참고 문헌

References

- [1] V Alfred, S Monica, Sethi Ravi, Ullman Jeffrey D, et al. *Compilers Principles, Techniques*. Pearson, 2007.
- [2] Cristiano Calcagno and Dino Distefano. Infer: An automatic program verifier for memory safety of c programs. In *NASA Formal Methods Symposium*, pages 459–465. Springer, 2011.
- [3] Patrick Cousot. *Principles of abstract interpretation*. MIT Press, 2021.
- [4] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 238–252, 1977.
- [5] Gary A Kildall. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 194–206, 1973.
- [6] Ted Kremenek. Finding software bugs with the clang static analyzer. *Apple Inc*, 8:2008, 2008.
- [7] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.
- [8] Jay Lee, Joongwon Ahn, and Kwangkeun Yi. React-trace: A semantics for understanding react hooks: An operational semantics and a visualizer for clarifying react hooks. *Proc. ACM Program. Lang.*, 9(OOPSLA2), October 2025.
- [9] Anders Møller and Michael I Schwartzbach. Static program analysis. *Notes. Feb*, 2012.



- [10] Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.
- [11] Henry Gordon Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society*, 74(2):358–366, 1953.
- [12] Xavier Rival and Kwangkeun Yi. *Introduction to static analysis: an abstract interpretation perspective*. MIT Press, 2020.
- [13] Go Team. go/parser. <https://pkg.go.dev/go/parser>, 2014.
- [14] Alan Mathison Turing et al. On computable numbers, with an application to the entscheidungsproblem. *J. of Math*, 58(345-363):5, 1936.

5 부록

5.1 사용자 메뉴얼

TraceInspector 웹 인터페이스에서 분석 대상 Go 소스 파일을 Open을 통해 분석을 요청한다. 이후 인터페이스는 업로드한 소스코드와 이에 대응되는 CFG, 그리고 오류 발생 여부를 표시한다.

사용자는 이후 하단의 재생 바를 드래그하여 각 분석 단계별 요약 프로그램 상태를 CFG 상에서 관측할 수 있으며, 좌/우 방향키를 이용하여 한 단계별 이동을 할 수 있다.

CFG는 각 함수 단위로 생성되며, 함수별 명령문과, 명령문이 실행될 프로그램 상태를 표시한다. 현 분석 단계에서 갱신된 프로그램 상태를 붉은색으로 음영 처리되어 변화를 직접 확인할 수 있다.

CFG 탐색 창은 드래그와 스크롤을 통해 이동 및 확대가 가능하며, 더블클릭을 통해 상태를 초기화할 수 있다.

마지막으로 재생 바 하단의 노란색 화살표는 오류 상태가 최초로 탐지된 분석 단계를 나타낸다. 화살표를 클릭하면 해당 분석 상태로 이동하고, 오류 정보와 더불어 어떤 명령의 어떤 변수 상태에서 오류가 발생하게 되었는지 관찰 가능하다.

5.2 운영자 메뉴얼

5.3 배포 가이드

5.4 Imp 형식 의미론

해당 부록은 Imp에 대한 big-step operational semantics의 일부를 수록한다.

프로그램 상태 $\sigma : \mathbb{V} \rightarrow \mathbb{Z}$ 는 각 변수를 값으로 대응시킨다.



식(expression)의 경우 프로그램 상태 σ 에서 식 e 를 실행했을 때 값 v 로 계산됨을 $\sigma \vdash e \Downarrow v$ 로 표현한다.

함수의 경우 각 함수 이름을 인자명들과 함수 몸통 명령문의 쌍으로 대응시키는 전역 함수 대응 $F : FunName \rightarrow (\mathbb{V}^*, S)$ 로 표현된다.

$$\sigma \vdash x \Downarrow \sigma(x) \text{ E-VAR}$$

$$\sigma \vdash n \Downarrow n \text{ E-INT}$$

$$\sigma \vdash b \Downarrow b \text{ E-BOOL}$$

$$\sigma \vdash s \Downarrow s \text{ E-STRING}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 + e_2 \Downarrow n_1 + n_2} \text{ E-ADD}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 - e_2 \Downarrow n_1 - n_2} \text{ E-SUB}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 * e_2 \Downarrow n_1 n_2} \text{ E-MUL}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 / e_2 \Downarrow n_1 / n_2} \text{ E-DIV}$$

$$\frac{\sigma \vdash e \Downarrow n}{\sigma \vdash -e \Downarrow -n} \text{ E-NEG}$$

$$\frac{\sigma \vdash e_1 \Downarrow v_1 \quad \sigma \vdash e_2 \Downarrow v_2}{\sigma \vdash e_1 = e_2 \Downarrow (v_1 = v_2)} \text{ E-EQ}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 < e_2 \Downarrow (n_1 < n_2)} \text{ E-LT}$$

$$\frac{\sigma \vdash e_1 \Downarrow b_1 \quad \sigma \vdash e_2 \Downarrow b_2}{\sigma \vdash e_1 \wedge e_2 \Downarrow (b_1 \wedge b_2)} \text{ E-AND}$$

$$\frac{\sigma \vdash e_1 \Downarrow b_1 \quad \sigma \vdash e_2 \Downarrow b_2}{\sigma \vdash e_1 \vee e_2 \Downarrow (b_1 \vee b_2)} \text{ E-OR}$$

$$\frac{\sigma \vdash e \Downarrow b}{\sigma \vdash \neg e \Downarrow \neg b} \text{ E-NOT}$$

$$\frac{\sigma \vdash e_1 \Downarrow a \quad \sigma \vdash e_2 \Downarrow i}{\sigma \vdash e_1[e_2] \Downarrow a[i]} \text{ E-INDEX}$$

$$\frac{\sigma \vdash e \Downarrow a}{\sigma \vdash \text{len}(e) \Downarrow |a|} \text{ E-LEN}$$

$$\frac{\sigma \vdash e_1 \Downarrow v_1 \quad \dots \quad \sigma \vdash e_n \Downarrow v_n \quad F(f) = (x_1, \dots, x_n, S) \quad \langle S, [x_1 \mapsto v_1, \dots, x_n \mapsto v_n] \rangle \Downarrow \langle \sigma', \text{return}(v) \rangle}{\sigma \vdash f(e_1, \dots, e_n) \Downarrow v} \text{ E-CALL}$$

명령문 S 를 프로그램 상태 σ 에서 실행했을 때는 결과 상태 σ' 와 제어 흐름 결과 r 를 반환하며 $\langle S, \sigma \rangle \Downarrow \langle \sigma', r \rangle$ 로 표기한다.

제어 흐름 결과는 break, continue, return를 처리하기 위한 내부 상태이며 normal, break, continue, return(v)로 정의된다.



$$\frac{\langle \epsilon, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle \text{ SEQ-NIL} \quad \frac{\langle s, \sigma \rangle \Downarrow \langle \sigma_1, \text{normal} \rangle \quad \langle S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ SEQ-CONS}}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle} \text{ SEQ-ABORT}$$

$$\frac{\langle s, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle \quad r \neq \text{normal}}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle} \text{ SEQ-ABORT} \quad \langle \text{skip}, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle \text{ S-SKIP}$$

$$\frac{\sigma \vdash e \Downarrow v}{\langle x := e, \sigma \rangle \Downarrow \langle \sigma[x \mapsto v], \text{normal} \rangle} \text{ S-ASSIGN}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S_t, \sigma \rangle \Downarrow \langle \sigma', r \rangle}{\langle \text{if } e \text{ then } S_t \text{ else } S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle} \text{ S-IFTRUE}$$

$$\frac{\sigma \vdash e \Downarrow \text{false} \quad \langle S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle}{\langle \text{if } e \text{ then } S_t \text{ else } S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle} \text{ S-IFFALSE}$$

$$\frac{\sigma \vdash e \Downarrow \text{false}}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle} \text{ S-WHILEFALSE}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma_1, \text{normal} \rangle \quad \langle \text{while } e \text{ do } S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ S-WHILETRUE}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma', \text{break} \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma', \text{normal} \rangle} \text{ S-WHILEBREAK}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma_1, \text{continue} \rangle \quad \langle \text{while } e \text{ do } S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ S-WHILECONTINUE}$$

$$\langle \text{break}, \sigma \rangle \Downarrow \langle \sigma, \text{break} \rangle \text{ S-BREAK} \quad \langle \text{continue}, \sigma \rangle \Downarrow \langle \sigma, \text{continue} \rangle \text{ S-CONTINUE}$$

$$\frac{\sigma \vdash e \Downarrow v}{\langle \text{return } e, \sigma \rangle \Downarrow \langle \sigma, \text{return}(v) \rangle} \text{ S-RETURN}$$

5.4.1 요약 도메인 Go 인터페이스

TraceInspector분석기는 Go의 generics를 활용하여 특정 요약도메인 구현에 의존하지 않



```
type AbstractAnalyzer [IntDom domain.IntegerDomain [IntDom], ArrDom ArrayDomain [IntDom, ArrDom]] struct {
    Function_cfgs      FunctionCFGMap
    Function_defs      imp.ImpFunctionMap
    Settings            AnalysisSettings
    Intdomain_default  IntDom
    Arraydomain_default ArrDom
    function_pre_mem_map map[imp.ImpFunctionName]*AbstractFunctionMem [IntDom, ArrDom]
    output_handler     *AnalyzerOutputHandler
}
```

그림 16: AbstractAnalyzer 구조체 정의와 generics를 활용한 모듈러한 설계

고, 사용자가 정의한 임의의 요약도메인을 활용할 수 있도록 구현하였다. AbstractDomain 인터페이스를 설계하여 정수 및 배열 타입에 대한 요약 도메인을 정의하고 분석기에 바로 활용할 수 있도록 구현되어 있다.

코드 16는 Tracelnspector분석기를 초기화하는 구조체의 정의이다. IntDom, ArrDom에 IntegerDomain, ArrayDomain 를 만족하는 도메인 구현체를 넘겨주면 분석기는 구현에 구현받지 않고 온전하 동작할 수 있다.

```
type AbstractDomain [DomainImpl any] interface {
    IsBot () bool
    IsTop () bool
    CreateBot () DomainImpl
    CreateTop () DomainImpl
    Incl (DomainImpl) bool
    Join (DomainImpl) (DomainImpl, bool)
    Widen (DomainImpl) DomainImpl
    String () string
    Clone () DomainImpl
}

type IntegerDomain [DomainImpl any] interface {
    AbstractDomain [DomainImpl]
    From_IntLitExpr (imp.IntLitExpr) DomainImpl
    Add (DomainImpl) DomainImpl
    Sub (DomainImpl) DomainImpl
    Mul (DomainImpl) DomainImpl
    Div (DomainImpl) DomainImpl
    Mod (DomainImpl) DomainImpl
    Eq (DomainImpl) BoolDomain
    Neq (DomainImpl) BoolDomain
    LessThan (DomainImpl) BoolDomain
    GreaterThan (DomainImpl) BoolDomain
    Leq (DomainImpl) BoolDomain
    Geq (DomainImpl) BoolDomain
    Neg () DomainImpl
    Filter (FilterQueryType, DomainImpl) DomainImpl
}
```

그림 17: AbstractDomain 과 IntegerDomain 인터페이스 정의

코드 17는 각각 공통 요약도메인과 정수 요약도메인의 추상 인터페이스 정의이다. AbstractDomain 는 모든 요약도메인 구현체들이 구현해야할 핵심 함수들을 정의하고 있고, IntegerDomain 는 산술 및 비교 연산 등 정수를 나타내는 도메인이 구현해야 할 함수들을 명세하고 있다. 이처럼 해당 함수들을 구현한 구현체를 Tracelnspector분석기는 그 구체적 형태에 대해 신경쓰지 않고 요약 실행의미론과 접목해서 실행할 수 있는 구조로 설계되었다.

주 의 문

2026년 다학제간캡스톤디자인I 최종보고서
TraceInspector

발행인 : 김신영, 강민성

발행일 : 2026년 6월 19일

발행처 : 국민대학교 소프트웨어융합대학 소프트웨어학부

주소 : (136-702) 서울시 성북구 정릉동 861-1

1. 이 보고서는 국민대학교 소프트웨어융합대학
소프트웨어학부에서 개설한 교과목
다학제간캡스톤디자인I 에서 수행한 프로젝트
최종보고서 입니다.
 2. 이 보고서의 내용은 국민대학교 소프트웨어학부 및 위
발행인들의 서면 허락없이 사용되거나, 재가공 될 수
없습니다.
-